AEGISAI ENTERPRISE EDITION

SUITE DE DOCUMENTACIÓN TÉCNICA

Manual de Instalación y Despliegue

Guía Autocontenida para Cloud Run, Docker Read-Only (C04.1) y Perímetro IAP (C04.3)

Versión de Plataforma:	**2.5.0 Enterprise Edition**
Alcance Normativo:	**OWASP AISVS v1.0, ISO/IEC 42001:2023, Google Cloud Security**
Licencia:	**Apache License 2.0 (Open Source)**

AEGIS AI ASSURANCE PLATFORM

Manual de Instalación y Despliegue en Producción

Versión: 2.5.0 Enterprise (Edición 2026)

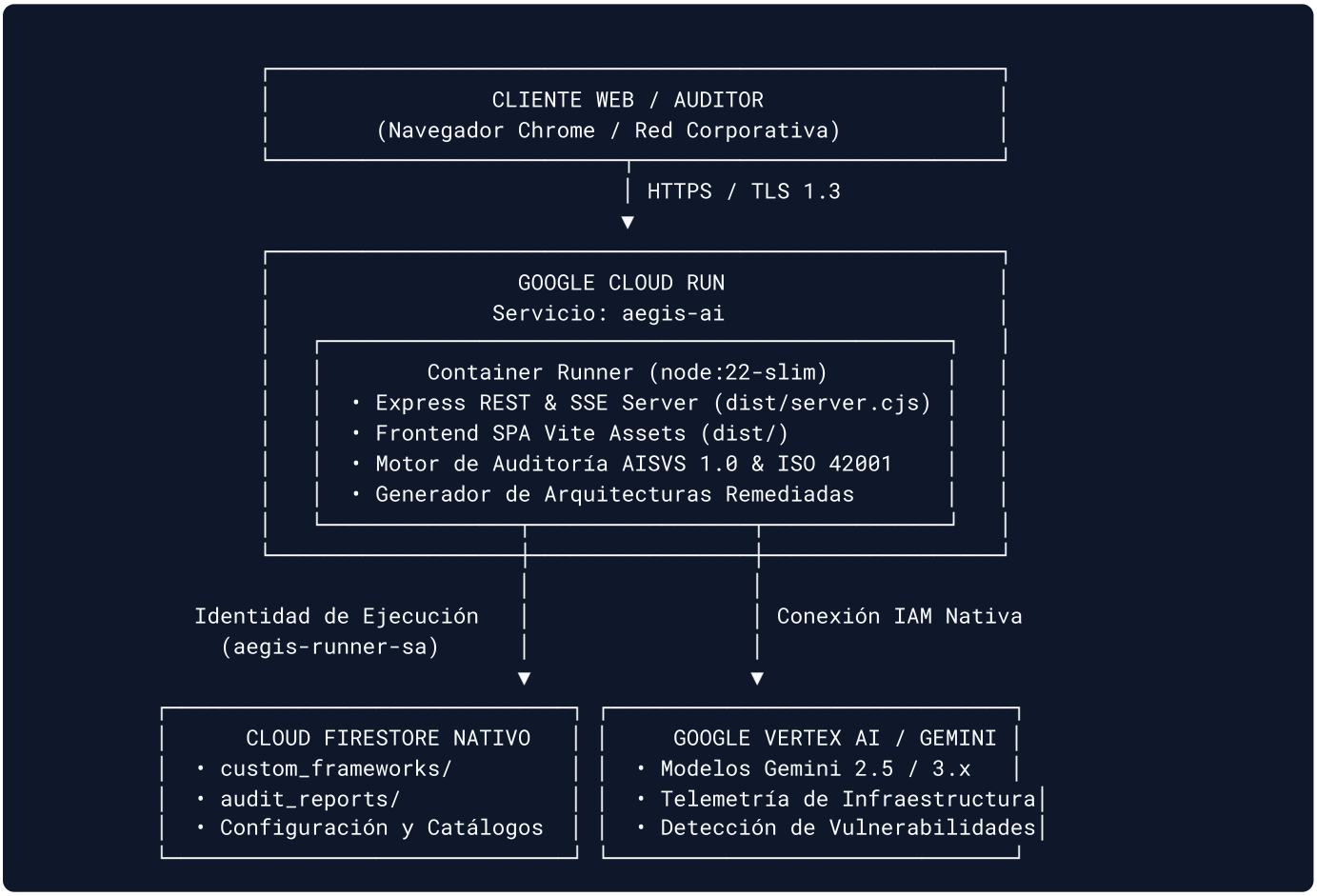
Clasificación: Guía Técnica de Infraestructura, Seguridad Cloud y DevOps

Plataformas Principales: Google Cloud Platform (Cloud Run, Firestore Native, Vertex AI, Artifact Registry, Firebase Auth)

Plataformas Alternativas: Docker Engine, Kubernetes, Servidores Linux Bare-Metal/VM

1. Resumen Ejecutivo y Arquitectura de Despliegue

La plataforma **AegisAI** está diseñada bajo un paradigma **Cloud-Native Serverless** de alta resiliencia y escalabilidad elástica en Google Cloud Platform (GCP). Su arquitectura desacopla el cómputo de la persistencia de datos y el gobierno de identidades:



2. Requisitos Previos y Herramientas

Para desplegar AegisAI en un proyecto corporativo de Google Cloud, el equipo de ingeniería debe contar con los siguientes elementos:

1. **Proyecto de Google Cloud**: Un proyecto activo con una cuenta de facturación (Cloud Billing) vinculada.
2. **Roles IAM Requeridos para el Desplegador**:
   - `roles/owner` o combinación de `roles/resourcemanager.projectIamAdmin`, `roles/run.admin`, `roles/artifactregistry.admin`, `roles/cloudbuild.builds.editor`, `roles/datastore.owner`.
3. **Google Cloud SDK (`gcloud` CLI)**:
   - Versión mínima: `gcloud >= 480.0.0`.
   - Se recomienda ejecutar el despliegue desde **Google Cloud Shell** (`https://shell.cloud.google.com`), el cual ya cuenta con `gcloud`, `docker`, `git` y `node` preconfigurados con autenticación nativa.
4. **Herramientas Locales (Opcional, si no se usa Cloud Shell)**:
   - `docker` >= 24.0.
   - `node` >= 20.10.0 y `npm` >= 10.0.0.

---

3. Método 1: Despliegue Rápido Automatizado (Recomendado)

AegisAI incluye el script de orquestación `deploy_cloud_run.sh` que automatiza la validación de dependencias, la habilitación de APIs, el aprovisionamiento de Service Accounts, la base de datos Firestore y la compilación del contenedor.

Pasos de Ejecución:

1. Abra **Google Cloud Shell** o su terminal autenticada en GCP.
2. Clone el repositorio o diríjase a la carpeta del proyecto:

```
git clone https://github.com/tu-organizacion/AegisAI.git
cd AegisAI
```

3. Otorgue permisos de ejecución al script:

```
chmod +x deploy_cloud_run.sh
```

4. Ejecute el script pasando el ID de su proyecto:

```
./deploy_cloud_run.sh ID_DE_TU_PROYECTO_GCP
```

5. El script ejecutará automáticamente las 12 fases de aprovisionamiento y devolverá la URL asignada:

```
🎉 ¡DESPLIEGUE EXITOSO EN GOOGLE CLOUD RUN!
🔗 URL del Servicio Cloud Run: https://aegis-ai-xxxxx-uc.a.run.app
```

---

4. Método 2: Despliegue Manual Paso a Paso (Para DevOps y SysAdmins)

Si las políticas corporativas exigen que cada comando sea auditado o aprovisionado mediante pipelines CI/CD (Terraform, Cloud Build Triggers, Jenkins o GitHub Actions), siga este procedimiento secuencial:

4.1 Configurar el Contexto del Proyecto

```
export PROJECT_ID="your-gcp-project-id"
export REGION="us-central1"
export SERVICE_NAME="aegis-ai"
export REPO_NAME="cloud-run-source-deploy"

gcloud config set project "$PROJECT_ID"
```

4.2 Habilitar las 7 APIs Requeridas de Google Cloud

AegisAI interactúa con servicios de cómputo, IA, compilación y base de datos. Habilite las APIs indispensables:

```
gcloud services enable \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  aiplatform.googleapis.com \
  firestore.googleapis.com \
  firebase.googleapis.com \
  identitytoolkit.googleapis.com \
  --project="$PROJECT_ID"
```

4.3 Configuración de Permisos IAM del Pipeline de Compilación

Para que Cloud Build pueda almacenar fuentes y escribir imágenes en Artifact Registry sin errores de permisos:

```
PROJECT_NUMBER=$(gcloud projects describe "$PROJECT_ID" --format="value(projectNumber)")
COMPUTE_SA="${PROJECT_NUMBER}-compute@developer.gserviceaccount.com"
CLOUDBUILD_SA="${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com"

# Asignar roles a Compute SA (utilizada por Cloud Build en entornos modernos)
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/cloudbuild.builds.builder" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/storage.admin" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/logging.logWriter" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${COMPUTE_SA}" \
  --role="roles/artifactregistry.admin" --quiet

# Asignar roles a Cloud Build Service Agent
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${CLOUDBUILD_SA}" \
  --role="roles/cloudbuild.builds.builder" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${CLOUDBUILD_SA}" \
  --role="roles/storage.admin" --quiet
```

4.4 Crear la Cuenta de Servicio de Ejecución con Mínimo Privilegio (`C05.2`)

Por seguridad Zero-Trust, el contenedor en Cloud Run **no debe ejecutarse con la cuenta predeterminada de Compute** ni recibir roles amplios de lectura de proyecto (`roles/viewer`). Se crea una cuenta con mínimo privilegio estricto:

```
SA_NAME="aegis-runner-sa"
SA_EMAIL="${SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"

gcloud iam service-accounts create "${SA_NAME}" \
  --description="Identidad de ejecucion con minimo privilegio para AegisAI Cloud Run" \
  --display-name="AegisAI Runner" \
  --project="$PROJECT_ID"

# Asignar UNICAMENTE roles de Vertex AI y Firestore (C05.2)
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/aiplatform.user" --quiet

gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${SA_EMAIL}" \
  --role="roles/datastore.user" --quiet
```

4.5 Aprovisionar Cloud Firestore en Modo Nativo

AegisAI almacena catálogos regulatorios personalizados y reportes de auditoría en Firestore:

```
# Verificar o crear la base de datos en modo Nativo
gcloud firestore databases create \
  --database="aegis-ai-db" \
  --location="nam5" \
  --type="firestore-native" \
  --project="$PROJECT_ID" --quiet || echo "Base de datos ya existe."
```

4.6 Configurar Firebase Authentication y Google Sign-In

Para que los usuarios inicien sesión de forma segura:

1. Ingrese a [Firebase Console](https://console.firebase.google.com/project/_/authentication/providers).
2. Seleccione su proyecto (`$PROJECT_ID`).
3. En la sección **Sign-in method** (Método de acceso), seleccione el proveedor **Google**:
   - Active el interruptor de habilitación.
   - Seleccione el correo de soporte corporativo del proyecto.
   - Haga clic en **Guardar**.
4. En [Firebase Console > Project Settings](https://console.firebase.google.com/project/_/settings/general), en la sección **Tus apps**, registre una app Web ( ) llamada `AegisAI` .
5. Guarde la configuración resultante en el archivo `firebase-applet-config.json` :

```
{
  "projectId": "your-gcp-project-id",
  "appId": "1:XXXXXXXXXX:web:XXXXXXXXXXX",
  "apiKey": "AIzaSyXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "authDomain": "your-gcp-project-id.firebaseapp.com",
  "firestoreDatabaseId": "aegis-ai-db",
  "storageBucket": "your-gcp-project-id.firebasestorage.app",
  "messagingSenderId": "XXXXXXXXX"
}
```

4.7 Compilar la Imagen de Contenedor sin Capas Obsoletas

Asegúrese de compilar con la bandera `--no-cache` para garantizar que los artefactos TypeScript y Vite queden actualizados:

```
# 1. Crear repositorio en Artifact Registry si no existe
gcloud artifacts repositories create "$REPO_NAME" \
  --repository-format=docker \
  --location="$REGION" \
  --description="Repositorio de imagenes AegisAI" \
  --project="$PROJECT_ID" --quiet || true

# 2. Compilar y subir la imagen
IMAGE_URL="${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO_NAME}/${SERVICE_NAME}:latest"

docker build --no-cache -t "$IMAGE_URL" .
docker push "$IMAGE_URL"
```

(Si no dispone de Docker localmente, puede compilar directamente en la nube con: `gcloud builds submit --tag "$IMAGE_URL" .`).

4.8 Desplegar en Google Cloud Run con Perímetro Zero-Trust IAP y VPC Egress (`C04.3`, `C05.3`)

```
gcloud beta run deploy "$SERVICE_NAME" \
  --image "$IMAGE_URL" \
  --project "$PROJECT_ID" \
  --region "$REGION" \
  --platform managed \
  --service-account "${SA_EMAIL}" \
  --min-instances 0 \
  --max-instances 10 \
  --memory 1Gi \
  --cpu 1 \
  --timeout 300s \
  --port 8080 \
  --set-env-vars "NODE_ENV=production,PORT=8080,PROJECT_ID=${PROJECT_ID},GOOGLE_CLOUD_LOCATION=${REG
  --set-secrets "ADMIN_API_TOKEN=aegis-admin-token:latest" \
  --vpc-connector "aegis-vpc-connector" \
  --vpc-egress "private-ranges-only" \
  --no-allow-unauthenticated \
  --iap
```

4.9 Registrar el Dominio Autorizado en Firebase

Una vez desplegado Cloud Run, obtenga la URL asignada:

```
SERVICE_URL=$(gcloud run services describe "$SERVICE_NAME" --project "$PROJECT_ID" --region "$REGION
echo "URL asignada: $SERVICE_URL"
```

1. Vaya a [Firebase Console > Authentication > Settings > Authorized domains] (https://console.firebase.google.com/project/_/authentication/settings).
2. Haga clic en **Add domain** (Agregar dominio).
3. Pegue el dominio de su servicio Cloud Run (ejemplo: `aegis-ai-123456789012.us-central1.run.app` o su dominio corporativo personalizado `aegisai.yourdomain.com`).

---

5. Control de Acceso Multi-Tenant, Filtro de Dominios Corporativos y Reglas Firestore (`C05.4`)

AegisAI aplica control de acceso en profundidad en tres capas (Cloud IAP, OAuth Frontend y Reglas Criptográficas de Firestore):

- **Dominios Permitidos Configurables**: Puede definir los dominios corporativos autorizados mediante `VITE_ALLOWED_DOMAINS` (por ejemplo `google.com,example.com`).
- **Validación Criptográfica de Tenant y Propietario (`C05.4`)**:
- En frontend, `Login.tsx` valida el dominio contra `VITE_ALLOWED_DOMAINS`, y `App.tsx` / `ApiConnectionView.tsx` vinculan `ownerUid`, `ownerEmail` y `tenantDomain` a cada reporte y clave de API creada.
- En base de datos, `firestore.rules` valida pertenencia a los dominios corporativos autorizados (`isAllowedDomain()`), propiedad en la creación (`hasValidOwnershipOnCreate()`) e inmutabilidad de propietario (`ownershipUnchanged()`):

```
    function isAllowedDomain() {
      return request.auth != null
        && request.auth.token.email is string
        && request.auth.token.email_verified == true
        && (
          request.auth.token.email.matches('.*@google\\.com$') ||
          request.auth.token.email.matches('.*@example\\.com$') ||
          request.auth.token.email.matches('.*@yourcompany\\.com$')
        );
    }
```

- Si un usuario intenta iniciar sesión con una cuenta no autorizada, la sesión se revoca de inmediato tanto en Cloud IAP como en Firestore.

---

6. Verificación Operativa y Pruebas de Humo (Smoke Tests)

Una vez completado el despliegue, verifique el correcto funcionamiento del servicio:

1. Comprobación del Endpoint de Salud

```
curl -I "${SERVICE_URL}/api/health"
```

Respuesta esperada:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"status":"ok"}
```

2. Comprobación de Cabeceras de Seguridad HTTP

```
curl -I "${SERVICE_URL}"
```

Verifique la presencia de las cabeceras corporativas:

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: SAMEORIGIN`
- `X-XSS-Protection: 1; mode=block`
- `Referrer-Policy: strict-origin-when-cross-origin`

3. Suite Automatizada de Pruebas de Integración

Desde la raíz del repositorio, puede ejecutar:

```
npm run test:integration
npm run test:gcp
npm run test:security
```

Todas las pruebas deben concluir con código de salida `0`.

---

7. Guía Forense de Solución de Problemas (Troubleshooting)

Esta sección documenta los incidentes más frecuentes encontrados en despliegues empresariales de GCP y su resolución paso a paso:

Incidente 1: `PERMISSION_DENIED` en Cloud Storage durante Cloud Build

- **Síntoma:** Al ejecutar `gcloud run deploy --source .` o `gcloud builds submit`, la terminal muestra:

```
ERROR: (gcloud.run.deploy) PERMISSION_DENIED: Build failed because the default service account
could not resolve source: Get "https://storage.googleapis.com/...": Access denied.
```

- **Causa:** La cuenta de servicio predeterminada de Compute (`{PROJECT_NUMBER}-compute@developer.gserviceaccount.com`) carece del rol de administrador de almacenamiento en el bucket de staging de Cloud Build.
- **Solución:**

```
PROJECT_NUMBER=$(gcloud projects describe "$PROJECT_ID" --format="value(projectNumber)")
gcloud projects add-iam-policy-binding "$PROJECT_ID" \
  --member="serviceAccount:${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \
  --role="roles/storage.admin"
```

Incidente 2: `Firebase: Error (auth/the-service-is-currently-unavailable.)`

- **Síntoma:** Al hacer clic en "Iniciar sesión con Google", la interfaz muestra un error rojo de servicio no disponible.
- **Causa:**

1. La API **Identity Toolkit** (`identitytoolkit.googleapis.com`) no está habilitada en el proyecto de GCP.
2. El archivo `firebase-applet-config.json` contiene llaves o identificadores de un proyecto sandbox antiguo que fue dado de baja.
   - **Solución:**

```
# 1. Habilitar la API
gcloud services enable identitytoolkit.googleapis.com --project="$PROJECT_ID"

# 2. Regenerar firebase-applet-config.json con los valores del proyecto activo
npx -y firebase-tools apps:sdkconfig WEB --project="$PROJECT_ID"
```

Incidente 3: `auth/unauthorized-domain` (Dominio no registrado)

- **Síntoma:** Al intentar autenticarse, aparece el mensaje:

```
El dominio de esta instancia aún no está registrado en los "Authorized Domains" de Firebase Aut
```

- **Causa:** El dominio generado por Cloud Run (`*.run.app`) no ha sido agregado a la lista blanca de OAuth en Firebase Authentication.
- **Solución:**

1. Copie el dominio de Cloud Run sin protocolo (ejemplo: `aegis-ai-123456789012.us-central1.run.app`).
2. Ingrese a **Firebase Console > Authentication > Settings > Authorized domains**.
3. Haga clic en **Add domain**, pegue el valor y guarde.

Incidente 4: `The user-provided container failed to start and listen on PORT=8080`

- **Síntoma:** El despliegue falla en la fase `Creating Revision...failed` y los registros del contenedor reportan `Container called exit(1)` .
- **Causa:**

1. En la etapa de producción del Dockerfile faltaba copiar `firebase-applet-config.json` o la carpeta `/config` .
2. Se importaron dependencias de desarrollo (`dotenv/config` o `vite`) de forma estática en el servidor de producción.
   - **Solución:**
3. Verifique que `Dockerfile` contenga: `COPY --from=builder /app/firebase-applet-config.json ./firebase-applet-config.json` .
4. Recompile obligando a Docker a no usar capas viejas en caché:

```
docker build --no-cache -t "$IMAGE_URL" .
docker push "$IMAGE_URL"
```

Incidente 5: `FAILED_PRECONDITION: One or more users named in the policy do not belong to a permitted customer`

- **Síntoma:** Al asignar `allUsers` o un dominio externo, `gcloud` arroja un error de violación de política organizacional.
- **Causa:** En organizaciones con la política *Domain Restricted Sharing* (`constraints/iam.allowedPolicyMemberDomains`), está bloqueado otorgar permisos a cuentas anónimas o ajenas a la organización.
- **Solución:**

Asigne el rol de invocador al dominio corporativo en lugar de `allUsers` :

```
gcloud run services add-iam-policy-binding "$SERVICE_NAME" \
  --project "$PROJECT_ID" \
  --region "$REGION" \
  --member="domain:yourdomain.com" \
  --role="roles/run.invoker"
```

Incidente 6: Error `403 Forbidden` al invocar `/api/evaluate` vía Firebase Hosting

- **Síntoma:** La página principal carga, pero al subir un archivo para evaluación la llamada a `/api/evaluate` falla con `403 Forbidden: Your client does not have permission to get URL /api/evaluate from this server`.
- **Causa:** Los rewrites de Firebase Hosting hacia Cloud Run solo transmiten peticiones a servicios públicos (`allUsers`). Si Cloud Run es privado, Firebase Hosting no inyecta tokens de identidad OIDC y la petición es denegada por Cloud IAM.
- **Solución:**

Acceda al servicio directamente a través de la URL de Cloud Run (`https://--.a.run.app`) o configure un **External Application Load Balancer con Cloud Run Backend** y certificados SSL corporativos.

---

8. Mantenimiento y Actualizaciones

Para desplegar nuevas versiones o actualizaciones de catálogos normativos:

```
# 1. Obtener los cambios del repositorio
git pull origin main

# 2. Validar tipos y ejecutar pruebas
npm run lint
npm run test:integration

# 3. Recompilar y desplegar nueva revisión
./deploy_cloud_run.sh "$PROJECT_ID"
```

Cloud Run gestionará la migración de tráfico de forma instantánea (**Zero-Downtime Deployment**) sin interrumpir las sesiones activas de los auditores.